Problem Situation - Evidence 2

Ethiel Favila Alvarado
Programming of Data Structures and Fundamental Algorithms
A00844789@tec.mx

I. WHAT IS THE PROBLEM, AND WHY IS IT RELEVANT?

The problem is that we need to identify, organize and analyze failed authentication attempts and unauthorized access requests logged in the system. It is important to analyze them properly because it is a security matter [1]. The unsorted security logs contain a lot of data, and without processing it is difficult to detect breaches or malicious IP addresses. Organizing them increases the chances of improving security.

II. HOW DID YOU SOLVE THE PROBLEM?

To solve the issue of the authentication reports being unordered and inaccessible, we focused on storing them safely in a way that lets us access the different parts of each log (timestamp, text, IP, etc.) and easily sort and filter them. First, we read the logs from bitacora.txt and store them in Keeper, a doubly linked list. Then, the Sorter class orders the list by IP address and, for the same IP, by time. After that, Searcher asks for a start and an end IP and extracts every record in that range. Since the list is already sorted, all the matching records are together, and the records of each IP are already in time order. Finally, the output is both printed and stored in output.txt.

The program has five modules plus main: Entry, Keeper, Sorter, Stack and Searcher. Entry is a struct that holds one log record: month, day, time, IP, message and its timestamp. Then we have the Node struct and the Keeper class. Node holds a previous and a next pointer, as well as one Entry object, so we can build a doubly linked list. Sorter runs the iterative quicksort, and StackNode (part of the Stack class) builds a singly linked list that replaces recursion in the quicksort. Finally, the Searcher class performs the range search, displays the results and stores them in output.txt.

III. DATA STRUCTURES AND ALGORITHMS

A. Which data structures and algorithms were used? How were they implemented, and why were they selected?

- *Doubly linked list*- Each log line is stored in an Entry (month, day, time, IP, message and timestamp). Each Entry goes in a node with next and prev pointers, and new nodes are added at the end with pushBack. It was used this way because we didn't know how many records we had originally, so the list grows with each record instead of reserving space beforehand.
- *Stack on a singly linked list*- The top of the stack is the head of the singly linked list. It has push and pop at the head, and each element has two pointers (low, high) that mark which part of the list still needs sorting. It was used this way because it replaces recursion in the quicksort, so we avoid overflowing the call stack with too many recursive calls.
- *Iterative quicksort*- Records are first ordered by IP and then by date and time. To compare IPs, each one (a.b.c.d) is turned into a base 1000 number (some octets in the log are greater than 255), so we compare real numbers instead of text. The last node of each partition is the pivot [2], and only the data of the nodes is swapped. It was used this way because quicksort is an efficient sort and, moreover, it sorts in place, so we don't need to copy the records (which are a lot) a second time like merge sort on an array would.
- *Range search*- We ask for the start and end IPs before searching; if start > end, we just switch them (a safety in case the user gives them in the wrong order). Then we skip nodes until we find the start IP and start collecting matches, and as soon as we pass the end IP we stop. We did it this way because the linked list has no random access, so binary search doesn't really work. The linear search, although basic, works in this case since the records are already sorted.

B. Is a linked list an appropriate data structure for this particular problem? Explain why or why not.

In our case a linked list works because we don't know the number of records. Since we don't know how many records we have, using pushBack helps grow the list dynamically, without ever having to resize or copy data. Furthermore, we can easily output the results after sorting: when Searcher finds the first and last record, we just store which ones they are and print and store whatever is between them, without having to copy anything (and thus avoiding wasted memory). Plus, insertion at the end and deletion of a known node are only O(1).

However, linked lists have deficiencies in this case too. First, we cannot use binary search because we cannot jump to the middle of the list; we have to compensate by walking from the head one node at a time, which costs more time. Plus, we use more memory since, besides the entry, we have two extra pointers per node.

C. What are the advantages and disadvantages of the data structures and algorithms used?

TABLE I
ADVANTAGES AND DISADVANTAGES

Structure / algorithm	**Advantages**	**Disadvantages**
Doubly linked list	It grows dynamically, so we don't need to know the number of records from the beginning. Records can be swapped without affecting the other records. Prev allows us to easily use quicksort.	We don't have random access, so we can't use binary search. Besides, it uses extra memory since we have two pointers per node.

Stack	We can add as many pending partitions as we need without it overflowing.	Each element needs an extra next pointer, which wouldn't happen with an array.
Iterative quicksort	It sorts in place: data is only swapped between nodes. The partition only needs sequential access (not random access), which fits the linked list. It's fast, and the iterative version doesn't overflow the call stack.	Its worst case is O(n^2), which depends on the order of the input (for example, a list that is already sorted). Merge sort guarantees O(n log n), which quicksort does not, so in the worst case quicksort can be much slower.
Range search	It is simple, so it's difficult to get it wrong. Since the information is already sorted, it stops early. We obtain a first and last pointer without complications.	Its worst case is O(n), because it has to walk node by node; binary search on an array would find the start IP in O(log n). In our case it works because everything is already sorted, but in general this is a disadvantage.

D. What are their best- and worst-case computational complexities, and what determines those cases?

TABLE II
BEST CASE AND WORST-CASE COMPLEXITIES OF ALL OPERATIONS

Algorithm	**Operation**	**Best case**	**Worst case**	**What determines the case?**
Doubly linked list	pushBack	O(1)	O(1)	Always the same thanks to the tail pointer
	Read	O(n)	O(n)	Each of the n lines is read and added
	Destructor	O(n)	O(n)	Every node must be freed
Stack	Push	O(1)	O(1)	Always works with the top
	Pop	O(1)	O(1)	Always works with the top
	isEmpty	O(1)	O(1)	Always works with the top
Iterative quicksort	Quicksort	O(n log n)	O(n^2)	The best case comes from the pivot landing in the middle every time, so the list is split in half. The worst case is when the pivot is always the smallest or largest value; since the pivot is the last node, this happens when the list is already sorted or reversed.
Range search	Search	O(1)	O(n)	Best case: the end IP is smaller than the first IP of the list, so it stops at the first node. Worst case: the range reaches the end of the list, so every node is visited.

IV. IMPROVEMENTS

The code works and efficiently manages all the records; however, we could make some improvements. First, we could try to use merge sort instead of quicksort. Merge sort can also work on linked lists, but it guarantees O(n log n) time regardless of the pivot. We could also handle more cases: in bitacora.txt all logs have the same format, but we don't account for corrupted lines when parsing.

An interesting possible feature would be repeated searches, allowing the user to run several queries without having to restart the program. Besides, the whole point of organizing the logs is to detect failed attempts, so it would be nice to flag IPs that have many failed attempts in a short time (which is easy because the attempts of each IP are already ordered by time). A filter not just by IP but also by attack time would be interesting too.

V. DIAGRAMS

DIAGRAM I
FLOW CHART OF THE PROGRAM IN GENERAL

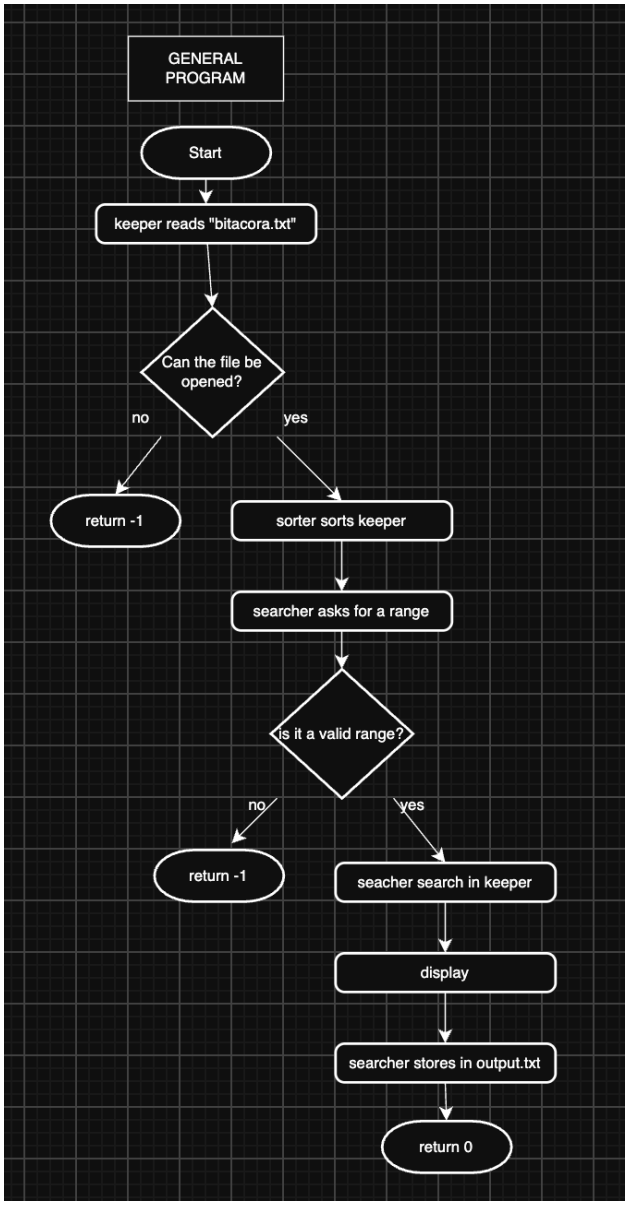


DIAGRAM II
HOW THE LINKED LIST IS CREATED

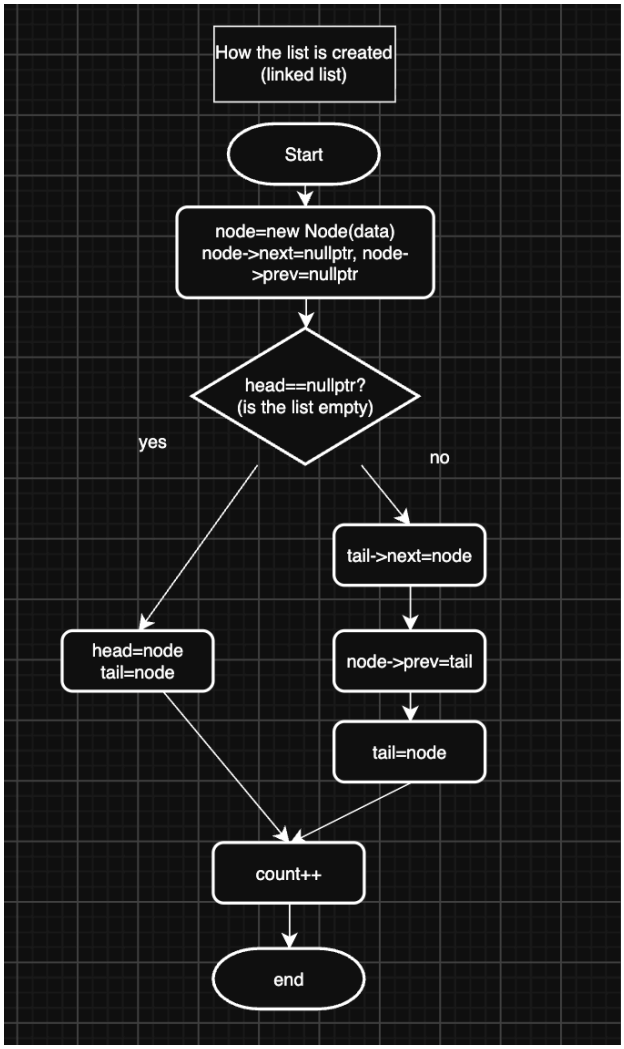


DIAGRAM III
HOW THE LINKED LIST IS SORTED

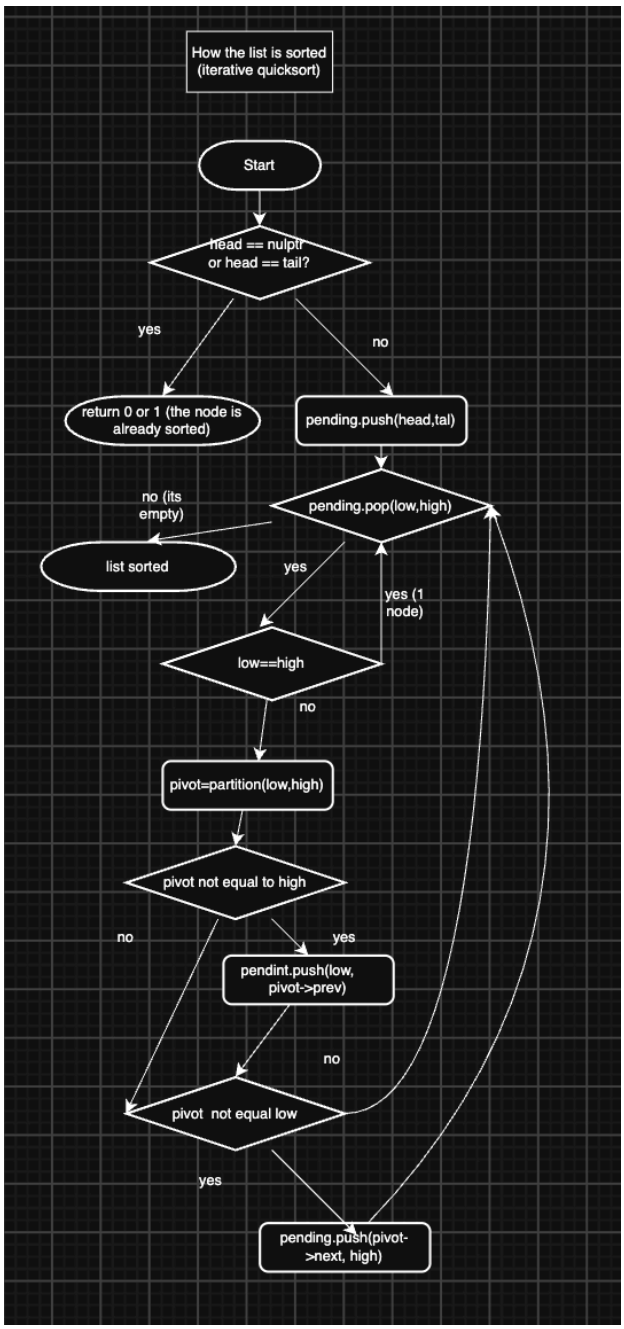
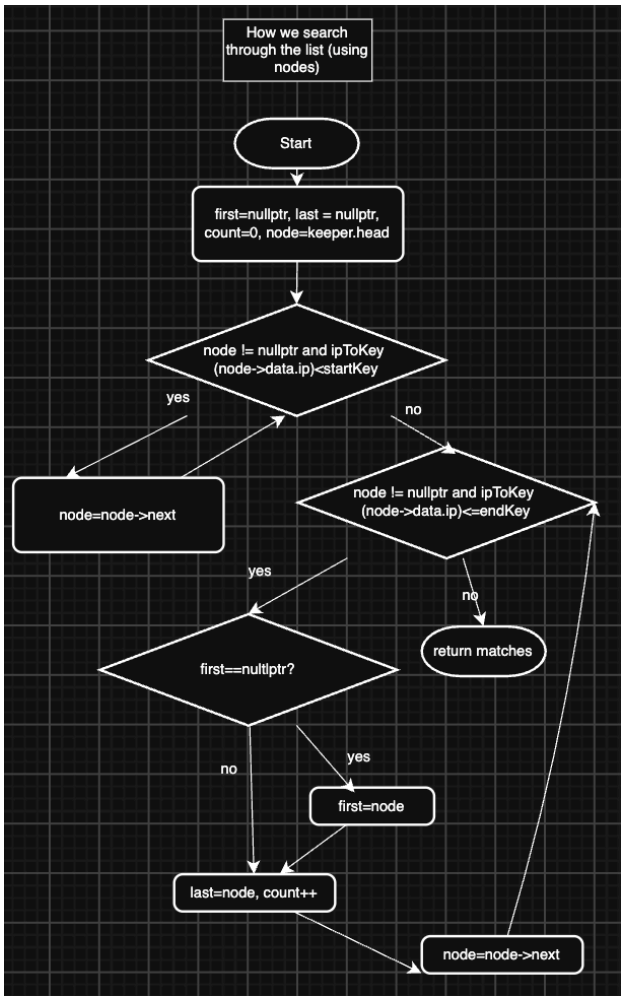


DIAGRAM IV
SEARCH METHOD



VI. REFERENCES

[1] OWASP Foundation, “Logging Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html [Accessed: Oct. 20, 2026].

[2] OpenDSA, “13.11. Quicksort,” OpenDSA Data Structures and Algorithms Modules. [Online]. Available: https://opendsa-server.cs.vt.edu/ODSA/Books/pubbook/odsa-all-0013bb43-a12f-4ab3-9b86-73b317f1e37e/summer-i-2018/Pub_Sum2018_All_ODSA/html/Quicksort.html [Accessed: Oct. 20, 2026].